Data source excerpt (continued) *data_source.py*

```
        raw_time, source_ip, payload, callsign_hint = _replay_entries[_replay_index]
        _replay_index += 1
        if _capture_logger is not None:
            _capture_logger.log_chunk(payload, source_ip, 0)
        decoded = _decoder.decode_payload(
            payload,
            source_ip=source_ip,
            timestamp=raw_time,
            callsign_hint=callsign_hint,
        )
        decoded_events.extend(decoded)
        if _capture_logger is not None:
            _capture_logger.log_decoded_many(decoded)
        processed += 1

    stale_seconds = None if SIMULATION_STALE_SECONDS <= 0 else SIMULATION_STALE_SECONDS
    raw_rows = _decoder.get_state_snapshot(stale_seconds=stale_seconds)
    return _clean_records(raw_rows), raw_rows, decoded_events


def _start_receiver_if_needed():
    global _receiver, _capture_logger
    if _receiver is not None:
        return

    _decoder.reset_state()
    _receiver = PingstationStreamReceiver(
        host=PINGSTATION_LISTEN_HOST,
        port=PINGSTATION_LISTEN_PORT,
        decoder=_decoder,
        stale_seconds=PINGSTATION_STALE_SECONDS,
        raw_logger=_capture_logger,
    )
    _receiver.start()


def get_aircraft_data():
    with _mode_lock:
        simulation_mode = SIMULATION_MODE

    if simulation_mode:
        cleaned_rows, raw_rows, decoded_events = _get_replayed_snapshot()
        return {
            "aircraft": cleaned_rows,
            "aircraft_raw": raw_rows,
            "aircraft_events": decoded_events,
        }

    _start_receiver_if_needed()
    live_rows = _receiver.get_snapshot() if _receiver else []
    live_events = _receiver.drain_records() if _receiver else []
    return {
        "aircraft": _clean_records(live_rows),
        "aircraft_raw": live_rows,
        "aircraft_events": live_events,
    }


def get_status():
    """Return a dict that describes the current data-source state.
    Called by the /api/status Flask route so the UI can show mode & health.
    """
    import time as _time

    with _mode_lock:
        simulation_mode = SIMULATION_MODE

    if simulation_mode:
        log_file = _get_simulation_log_file()
        return {
            "mode": "simulation",
            "simulationFile": log_file,
            "simulationFileExists": Path(log_file).exists(),
            "replayIndex": _replay_index,
            "replayTotal": len(_replay_entries),
            "configFile": str(_CFG_PATH),
            "sessionDir": str(SESSION_DIR),
            "hexFile": str(_capture_logger.hex_file_path) if _capture_logger else None,
            "asciiFile": str(_capture_logger.ascii_file_path) if _capture_logger else None,
            "rawFile": str(_capture_logger.raw_file_path) if _capture_logger else None,
        }

    # --- live mode ---
    _start_receiver_if_needed()
    conn_status = _receiver.get_connection_status() if _receiver else {}
    last_data = conn_status.get("lastDataTime")
    seconds_since_data = (
        round(_time.time() - last_data, 1) if last_data is not None else None
    )
    return {
        "mode": "live",
        "connected": conn_status.get("connected", False),
        "listenHost": conn_status.get("listenHost", PINGSTATION_LISTEN_HOST),
        "listenPort": conn_status.get("listenPort", PINGSTATION_LISTEN_PORT),
        "lastDataSecondsAgo": seconds_since_data,
        "totalFramesDecoded": conn_status.get("totalFramesDecoded", 0),
        "totalConnections": conn_status.get("totalConnections", 0),
        "bindError": conn_status.get("bindError"),
        "configFile": str(_CFG_PATH),
        "sessionDir": str(SESSION_DIR),
        "hexFile": str(_capture_logger.hex_file_path) if _capture_logger else None,
        "asciiFile": str(_capture_logger.ascii_file_path) if _capture_logger else None,
        "rawFile": str(_capture_logger.raw_file_path) if _capture_logger else None,
    }
```